

GTP3: Programación en C++ para Ciencia e Ingeniería

Ejercicios

Estanislao Claucich	1
La clase <code>board_t</code>	2
Jugador <code>dumb_t</code>	4
Jugador <code>random_t</code>	7
Jugador <code>agressive_t</code>	9
Jugador <code>minimax_t</code>	11
Jugador <code>rule_based_t</code>	15
Resultados	18

Estanislao Claucich

A continuación se presentan las soluciones a los ejercicios propuestos en el GTP3.

Consigna

Implementar un programa que simule el juego del ta-te-ti.

Se deben implementar diferentes tipos de jugadores que tendrán diferentes estrategias para elegir sus movimientos:

- `dumb_t`: Elige la primera celda vacía que encuentra.
- `random_t`: Elige una celda vacía al azar.
- `agressive_t`: Busca jugadas que lo hagan ganar o bloqueen al oponente.
- `minimax_t`: Implementa el algoritmo minimax para elegir la mejor jugada posible.
- `rule_based_t`: Implementa una estrategia basada en reglas para elegir la jugada.

La clase `board_t`

Esta clase será la encargada de representar el tablero del juego, y de verificar el estado del mismo (si hay un ganador, si el tablero está lleno, etc). Además, tendrá métodos para realizar movimientos y mostrar el tablero por pantalla.

El tablero se representará fácilmente con un `vector<char> b`, donde cada posición del vector representa una de las 9 casillas del tablero. Cada posición puede contener el valor `-1`, en caso de que haya jugado el jugador de las X, un `1`, en caso de que haya jugado el jugador de las O, o un `0` si la casilla está vacía.

Al mismo tiempo, se llevará registro de en qué ronda se realizó cada movimiento. Esto se realiza con un `vector<int> rounds`, donde cada posición del vector representa una de las 9 casillas del tablero, y el valor que contiene es la ronda en la que se realizó el movimiento en esa casilla. Si la casilla está vacía, el valor será `-1`.

El constructor de esta clase será entonces simplemente:

```
1 board_t() {  
2     b.assign(9,0);  
3     rounds.assign(9,-1);  
4 }
```

Y cuando se necesita limpiar el tablero, se tiene el método `clear()`, que hace exactamente lo mismo que el constructor:

```
1 void clear() {  
2     b.assign(9,0);  
3     rounds.assign(9,-1);  
4 }
```

La clase a su vez contiene un método `wins()`, que verifica si en ese momento hay un ganador. Para esto, se define un arreglo estático `lines[8][3]`, que contiene las combinaciones de casillas que representan una victoria (las 3 filas, las 3 columnas y las 2 diagonales). Esta variable se define de forma estática y global, ya que será útil en distintas partes de la lógica del juego. El método `wins()` recorre este arreglo, y para cada combinación de casillas, suma los valores de esas casillas en el vector `b`. Si la suma es `-3`, significa que el jugador de las X ha ganado, y si la suma es `3`, significa que el jugador de las O ha ganado. Si ninguna de las combinaciones da un resultado ganador, el método devuelve `0`.

```
1 static int lines[8][3] = {  
2     {0, 1, 2}, {3, 4, 5}, {6, 7, 8}, // Filas  
3     {0, 3, 6}, {1, 4, 7}, {2, 5, 8}, // Columnas
```

```

4         {0, 4, 8}, {2, 4, 6}           // Diagonales
5     };

1 int wins() {
2     for (int i = 0; i < 8; ++i) {
3         // Al usar -1 y +1, podemos sumar los valores directamente
4         int sum = b[lines[i][0]] + b[lines[i][1]] + b[lines[i][2]];
5
6         if (sum == -3) return -1; // Ganaron las cruces (-1)
7         if (sum == 3) return 1; // Ganaron los círculos (1)
8     }
9     // Si ninguna fila da una suma de +3 o -3,
10    // significa que todavía no hay ganador
11    return 0;
12 }

```

Para poder hacer que un jugador realice una jugada en alguna celda, se tiene el método `play(int player, int cell, int round=-1, bool verbose=true)`. Este método recibe como parámetros el jugador que va a realizar la jugada (que puede ser -1 para las X o 1 para las O), la celda en la que se va a realizar la jugada (un número entre 0 y 8, donde cada número representa una de las casillas del tablero), la ronda en la que se realiza la jugada, y un parámetro `verbose` que en caso de que sea `true`, muestra información adicional por consola.

Lo único que hace el método para guardar la jugada, es asignar el valor del parámetro `player` a la posición `cell` del tablero `b`. De esa forma se va llenando el tablero con las jugadas. A su vez, se actualiza de forma similar el vector `rounds`, asignando el valor del parámetro `round` a la posición `cell` del vector `rounds`. Así, vamos llevando un registro de en qué ronda se realizó cada jugada.

```

1 void play(int player, int cell, int round=-1, bool verbose=true){
2     if (verbose) {
3         cout << "Jugador " << (player == -1 ? "X" : "O") <<
4         " juega en la celda " << cell << endl;
5     }
6     b[cell] = player;
7     if (round != -1) rounds[cell] = round;
8 }

```

Esto básicamente establece la clase principal para definir el juego. Luego se definirán diferentes tipos de jugadores, con distintas estrategias que usarán el tablero definido previamente.

Jugador dumb_t

Este es el jugador más básico que se puede implementar. Su estrategia es simplemente elegir la primera celda vacía que encuentre en el tablero, sin importar si esa jugada le da una victoria o no, o si le permite bloquear una victoria del oponente.

i Jugadores

Todos los jugadores tendrán por lo menos dos métodos: `move(board_t &b, int me)` y `label()`. El método `move` es el encargado de realizar la jugada del jugador según sus reglas, y el método `label` devuelve el nombre del jugador.

En este caso, el método `move` es muy simple, se recorre el vector `b` del tablero, y se devuelve la primera posición que tenga un valor de 0, es decir, la primera celda vacía. Si no hay celdas vacías, se devuelve `-1` para indicar que no hay movimientos disponibles.

```
1 class dumb_t {
2     public:
3         int move(board_t &b, int me) {
4             for (int k=0; k<9; ++k) {
5                 if (b.b[k] == 0)
6                     return k; // Devuelve la primera celda vacía
7             }
8             return -1; // No hay movimientos disponibles
9         }
10
11         string label() { return "dumb";}
12     };
```

i Partido entre jugadores

Para efectivamente configurar un partido entre dos jugadores de tipo A y B, se implementa una función `match` que recibe como parámetros dos jugadores de los tipos correspondientes. Luego, la lógica de la función es independiente del tipo de jugador, ya que se llama al método `move` de cada jugador para obtener la jugada que quieren realizar, y se utiliza el método `play` del tablero para realizar la jugada. La función también verifica después de cada jugada si hay un ganador utilizando el método `wins` del tablero. Si hay un ganador, se devuelve el resultado del partido. Si no hay un ganador y el tablero está lleno, se devuelve 0 para indicar un empate.

La función, a su vez, permite recibir una cantidad de partidos a jugar. Esto permitirá realizar varias partidas bajo una cierta configuración para luego obtener estadísticas al respecto. El resultado de cada una de estas partidas se irá guardando en un vector de tres

elementos [victorias de X, victorias de O, empates], que se devolverá al finalizar todas las partidas.

En cada partida, lo primero que se hace es crear un `board_t` vacío y mientras que haya jugadas posible, se juega. El jugador de las X (-1) siempre va primero, y dependiendo del `current_player`, se llama al método `move` del jugador correspondiente para obtener la jugada que quiere realizar. Luego se llama al método `play` del tablero para realizar la jugada, y se verifica si hay un ganador. Si es así, se actualiza el vector de resultados y se termina la partida. Si no hay un ganador y el tablero está lleno, se actualiza el vector de resultados para indicar un empate y se termina la partida. Si no hay un ganador y el tablero no está lleno, se alterna el jugador actual y se continúa jugando.

```
1 vector<int> match(dumb_t &playX, dumb_t &playO, int n_matches=1, bool verbose=true) {
2     vector<int> results = {0, 0, 0}; // victorias de X, victorias de O, empates
3
4     for (int i=0; i<n_matches; ++i) {
5         srand(time(0) + i); // Cambiar la semilla para cada partida
6
7         board_t board;
8         int current_player = -1; // Cruces empieza
9         int round = 0;
10        while (true) {
11            int move;
12            if (current_player == -1) {
13                move = playX.move(board, current_player);
14            } else {
15                move = playO.move(board, current_player);
16            }
17
18            if (move == -1) {
19                results[2]++; // Empate
20                break;
21            }
22            board.play(current_player, move, round, verbose);
23            if (verbose) {
24                board.dump();
25            }
26            round++;
27
28            int winner = board.wins();
29            if (winner != 0) {
30                if (winner == -1) results[0]++; // Victoria de X
31                else results[1]++; // Victoria de O

```

```

32         break;
33     }
34
35     current_player *= -1; // Alternar jugador
36 }
37 }
38 return results;
39 }

```

En el caso de dos jugadores `dumb_t` enfrentándose entre sí, el resultado es completamente determinístico e igual en todas las partidas. El jugador de las **X** siempre gana, ya que elige la primera celda y terminará completando una de las diagonales.

A continuación se muestra la salida de un partido entre dos jugadores `dumb_t`:

```

bash

dumb vs dumb:
Jugador X juega en la celda 0
X . .
. . .
. . .
-----
Jugador O juega en la celda 1
X O .
. . .
. . .
-----
Jugador X juega en la celda 2
X O X
. . .
. . .
-----
Jugador O juega en la celda 3
X O X
O . .
. . .
-----
Jugador X juega en la celda 4
X O X
O X .
. . .
-----

```

Jugador O juega en la celda 5

X O X

O X O

...

Jugador X juega en la celda 6

X O X

O X O

X . .

Gana X

Jugador `random_t`

Este jugador también es muy básico. De todas las celdas vacías del tablero en el momento de jugar, elige una al azar. Para esto, primero se obtienen todas las celdas vacías en un vector `empty_cells`, y luego se elige una de esas celdas al azar utilizando la función `rand()`. Si no hay celdas vacías, se devuelve `-1` para indicar que no hay movimientos disponibles.

```
1 class random_t {
2     public:
3         int move(board_t &b, int me) {
4             vector<int> empty_cells;
5             for (int k=0; k<9; ++k) {
6                 if (b.b[k] == 0)
7                     empty_cells.push_back(k);
8             }
9
10            // No hay movimientos disponibles
11            if (empty_cells.empty()) return -1;
12            int random_index = rand() % empty_cells.size();
13            return empty_cells[random_index];
14        }
15
16        string label() { return "random";}
17    };
```

En el caso de que dos jugadores `random_t` se enfrenten entre sí, el resultado no es determinístico y puede variar en cada partida. Sin embargo, como se verá más adelante, el jugador que comienza la partida siempre parece tener una ventaja, ya que tiene más oportunidades de completar una línea antes de que el oponente pueda bloquearlo. Por lo tanto, es común observar

que el jugador que comienza (en este caso, las X) gana con mayor frecuencia que el jugador que va segundo (las O), aunque también pueden ocurrir muchos empates.

Si enfrentamos a un jugador `random_t` (X) contra un jugador `dumb_t` (O), es interesante ver que este último puede ser que gane, si tiene la suerte suficiente que las jugadas aleatorias del otro jugador no lleguen a bloquearlo.

```
bash
```

```
random vs dumb:
```

```
Jugador X juega en la celda 5
```

```
. . .  
. . X  
. . .
```

```
-----  
Jugador O juega en la celda 0
```

```
O . .  
. . X  
. . .
```

```
-----  
Jugador X juega en la celda 8
```

```
O . .  
. . X  
. . X
```

```
-----  
Jugador O juega en la celda 1
```

```
O O .  
. . X  
. . X
```

```
-----  
Jugador X juega en la celda 6
```

```
O O .  
. . X  
X . X
```

```
-----  
Jugador O juega en la celda 2
```

```
O O O  
. . X  
X . X
```

```
-----  
Gana O
```


Jugador `agresive_t`

Este jugador es un poco más especial, ya que tratará de buscar activamente si existe una jugada que lo haga ganar y, al mismo tiempo, buscar si el otro jugador está por ganar, para poder bloquearlo. Si no encuentra ninguna de estas dos situaciones, entonces elige una celda vacía al azar, como el jugador `random_t`. Esto ya representa un jugador que hace jugadas un poco más inteligentes que los dos anteriores, y como se puede ver en los resultados al final del documento, tiene una tasa de victorias mucho mayor que los jugadores anteriores.

i Método `counts()`

Este jugador utilizará un método de la clase `board_t` llamado `counts()`, que recibe dos vectores por referencia, `Xmarks` y `Omarks`, y los llena con la cantidad de marcas de cada jugador en cada una de las 8 líneas posibles (3 filas, 3 columnas y 2 diagonales). De esta forma, el jugador puede fácilmente verificar si hay alguna línea en la que tenga 2 marcas y una celda vacía, lo que le permitiría ganar, o si el oponente tiene 2 marcas y una celda vacía, lo que le permitiría bloquearlo.

```
1 void counts(vector<int> &Xmarks, vector<int> &Omarks){
2     // Pongo en 0 los contadores
3     Xmarks.assign(8, 0);
4     Omarks.assign(8, 0);
5
6     for(int i = 0; i < 8; ++i) {
7         for(int j = 0; j < 3; ++j) {
8             if(b[lines[i][j]] == -1) Xmarks[i]++;
9             else if(b[lines[i][j]] == 1) Omarks[i]++;
10        }
11    }
12 }
```

```
1 class agresive_t {
2     public:
3         int move(board_t &b, int me) {
4             // Obtengo la cantidad de marcas de cada jugador
5             vector<int> Xmarks, Omarks;
6             b.counts(Xmarks, Omarks);
7
8             int opponent = -me;
9
10            // Intentar ganar (si tengo una línea con 2 marcas más)
11            for (int i = 0; i < 8; ++i) {
```

```

12         if ((me == -1 && Xmarks[i] == 2) ||
13             (me == 1 && Omarks[i] == 2)) {
14             for (int j = 0; j < 3; ++j) {
15                 int cell = b.b[lines[i][j]];
16                 // Jugar en la celda vacía para ganar
17                 if (cell == 0) return lines[i][j];
18             }
19         }
20     }
21
22     // Intentar bloquear (si el oponente tiene una línea con 2 marcas)
23     for (int i = 0; i < 8; ++i) {
24         if ((opponent == -1 && Xmarks[i] == 2) ||
25             (opponent == 1 && Omarks[i] == 2)) {
26             for (int j = 0; j < 3; ++j) {
27                 int cell = b.b[lines[i][j]];
28                 // Jugar en la celda vacía para bloquear
29                 if (cell == 0) return lines[i][j];
30             }
31         }
32     }
33
34     // Si no hay jugada ganadora o bloqueadora, jugar aleatoriamente
35     vector<int> empty_cells;
36     for (int k=0; k<9; ++k) {
37         if (b.b[k] == 0)
38             empty_cells.push_back(k);
39     }
40     if (empty_cells.empty()) return -1;
41     int random_index = rand() % empty_cells.size();
42     return empty_cells[random_index];
43 }
44
45 string label() { return "agresive";}
46 };

```

Como se podrá ver en la sección de resultados, este es un jugador que presenta una estrategia mucho más ganadora que los dos jugadores que ya vimos. Sin embargo, no es una estrategia perfecta, ya que puede ser que el jugador `random_t` tenga la suerte de no bloquearlo en algún momento, o que el jugador `agresive_t` no logre detectar una jugada ganadora o bloqueadora en alguna situación particular. Por lo tanto, aunque este jugador tiene una tasa de victorias mucho mayor que los jugadores anteriores, todavía puede perder contra ellos en algunas

partidas.

Jugada de ejemplo entre un `agresive_t` (X) y un `random_t` (O):

```
bash

agresive vs random:
Jugador X juega en la celda 6
...
...
X..
-----
Jugador O juega en la celda 5
...
..O
X..
-----
Jugador X juega en la celda 3
...
X.O
X..
-----
Jugador O juega en la celda 7
...
X.O
XO.
-----
Jugador X juega en la celda 0
X..
X.O
XO.
-----
Gana X
```

Jugador `minimax_t`

En este caso, el jugador implementa el algoritmo minimax para elegir su jugada. Este algoritmo es un método de búsqueda utilizado en juegos de dos jugadores para determinar la mejor jugada posible, asumiendo que el oponente también juega de manera óptima. El jugador `minimax_t` evalúa todas las posibles jugadas futuras y asigna un valor a cada una de ellas, considerando tanto sus propias jugadas como las del oponente. Luego, elige la jugada que maximiza su probabilidad de ganar mientras minimiza la probabilidad de que el oponente gane. De esta forma, el jugador que siga esta estrategia siempre jugará de la mejor manera posible, y no

perderá contra ningún otro jugador, aunque sí puede empatar contra otro jugador que también siga esta estrategia, como se puede ver en la sección de resultados al final del documento.

Cabe destacar que esta estrategia es la más compleja de implementar, ya que requiere una función recursiva que evalúe todas las posibles jugadas futuras, y una función de evaluación que asigne un valor a cada estado del tablero. A su vez, es la más cara computacionalmente, ya que el número de posibles jugadas futuras crece exponencialmente a medida que se profundiza en el árbol de decisiones. Sin embargo, dado que el juego del ta-te-ti tiene un espacio de estados relativamente pequeño, es posible implementar esta estrategia sin problemas y obtener resultados en un tiempo razonable.

La función `phi` es la función recursiva que evalúa el estado del tablero y asigna un valor a cada jugada. Esta función verifica si hay un ganador en el estado actual del tablero, y devuelve 1 si el jugador actual ha ganado, -1 si el oponente ha ganado, o 0 si hay un empate. Si no hay un ganador, la función evalúa todas las posibles jugadas futuras, y devuelve el mejor valor para el jugador actual, asumiendo que el oponente también juega de manera óptima.

```
1 class minimax_t {
2     private:
3         int phi(board_t &b, int player, int playsnow) {
4             // Vemos si ya hay un ganador
5             int winner = b.wins();
6             if (winner == player) return 1;
7             if (winner == -player) return -1;
8
9             // Revisamos si hay celdas vacías para seguir jugando
10            bool has_empty = false;
11            for (int i = 0; i < 9; ++i) {
12                if (b.b[i] == 0) {
13                    has_empty = true;
14                    break;
15                }
16            }
17            // Si no hay celdas vacías, empate
18            if (!has_empty) return 0;
19
20            // Inicializamos el mejor puntaje dependiendo de si es
21            // nuestro turno (buscamos maximizar),
22            // o del oponente (buscamos minimizar)
23            int best_score = (playsnow == player) ? -1000 : 1000;
24
25            for (int k = 0; k < 9; ++k) {
26                if (b.b[k] == 0) {
```

```

27         // Hacemos una jugada temporal para probarla
28         b.b[k] = playsnow;
29         // Calculamos el puntaje con esta jugada hecha
30         int score = phi(b, player, -playsnow);
31         // Borrarnos la jugada
32         b.b[k] = 0;
33
34         if (playsnow == player) {
35             // Maximizamos para nuestro jugador
36             best_score = max(best_score, score);
37         } else {
38             // Minimizamos para el oponente
39             best_score = min(best_score, score);
40         }
41     }
42 }
43
44 return best_score;
45 }
46
47 public:
48     int move(board_t &b, int me) {
49         int best_score = -1000;
50         int best_move = -1;
51
52         for (int k = 0; k < 9; ++k) {
53             if (b.b[k] == 0) {
54                 b.b[k] = me;
55                 int score = phi(b, me, -me);
56                 b.b[k] = 0;
57
58                 // Maximizamos el resultado en nuestra primera jugada
59                 if (score > best_score) {
60                     best_score = score;
61                     best_move = k;
62                 }
63             }
64         }
65         return best_move;
66     }
67
68     string label() { return "minimax";}

```

Ejemplo de partida entre `minimax_t` (X) y `agressive_t` (O):

```
bash
```

```
minimax vs aggressive:
```

```
Jugador X juega en la celda 0
```

```
X . .
```

```
. . .
```

```
. . .
```

```
-----
```

```
Jugador O juega en la celda 8
```

```
X . .
```

```
. . .
```

```
. . O
```

```
-----
```

```
Jugador X juega en la celda 2
```

```
X . X
```

```
. . .
```

```
. . O
```

```
-----
```

```
Jugador O juega en la celda 1
```

```
X O X
```

```
. . .
```

```
. . O
```

```
-----
```

```
Jugador X juega en la celda 6
```

```
X O X
```

```
. . .
```

```
X . O
```

```
-----
```

```
Jugador O juega en la celda 3
```

```
X O X
```

```
O . .
```

```
X . O
```

```
-----
```

```
Jugador X juega en la celda 4
```

```
X O X
```

```
O X .
```

```
X . O
```

```
-----
```

Jugador rule_based_t

En este caso también vemos un jugador que es muy efectivo, pero sin necesidad de ser tan complejo como el caso de `minimax_t`. Al ser el ta-te-ti un ejemplo bastante simple, es posible implementar una estrategia que sea casi tan efectiva como el minimax, pero sin necesidad de evaluar todas las jugadas futuras. Esta estrategia se basa en un conjunto de reglas que el jugador sigue para elegir su jugada. Por ejemplo, el jugador puede seguir las siguientes reglas:

1. Si hay una jugada que le da la victoria, hacerla.
2. Si el oponente tiene una jugada que le da la victoria, bloquearla.
3. Si el centro del tablero está vacío, tomarlo.
4. Si el oponente ha jugado en una esquina, jugar en la esquina opuesta.
5. Si hay una esquina vacía, jugar en una esquina.
6. Si hay un borde vacío, jugar en un borde.

En vez de realizar el árbol de todas las posibles jugadas futuras, se reduce el espacio de búsqueda a situaciones muy particulares que pueden ocurrir y el jugador toma decisiones en función de la situación actual del tablero. Como se puede ver en los resultados al final del documento, esta estrategia es casi tan efectiva como el minimax, y es mucho más rápida de ejecutar, ya que no requiere evaluar todas las jugadas futuras.

```

1 class rule_based_t {
2     public:
3         int move(board_t &b, int me) {
4             vector<int> Xmarks, Omarks;
5             b.counts(Xmarks, Omarks);
6             int opponent = -me;
7
8             //Si hay una jugada ganadora, la juega.
9             for (int i = 0; i < 8; ++i) {
10                 if ((me == -1 && Xmarks[i] == 2) ||
11                     (me == 1 && Omarks[i] == 2)) {
12                     for (int j = 0; j < 3; ++j) {
13                         int cell = b.b[lines[i][j]];
14                         // Jugar en la celda vacía para ganar

```

```

15         if (cell == 0) return lines[i][j];
16     }
17 }
18 }
19
20 // Intentar bloquear al oponente
21 for (int i = 0; i < 8; ++i) {
22     if ((opponent == -1 && Xmarks[i] == 2) ||
23         (opponent == 1 && Omarks[i] == 2)) {
24         for (int j = 0; j < 3; ++j) {
25             int cell = b.b[lines[i][j]];
26             // Jugar en la celda vacía para bloquear
27             if (cell == 0) return lines[i][j];
28         }
29     }
30 }
31
32 //Juega alguna celda libre en el siguiente orden:
33 // Si esta libre el centro, lo toma.
34 if (b.b[4] == 0) return 4;
35
36 // Posiciones de esquinas y sus opuestas
37 int corners[4] = {0, 2, 6, 8};
38 int opp_corners[4] = {8, 6, 2, 0};
39
40 // Si el oponente tiene una esquina y la opuesta esta libre, la toma.
41 for (int i = 0; i < 4; ++i) {
42     if (b.b[corners[i]] == opponent && b.b[opp_corners[i]] == 0) {
43         return opp_corners[i];
44     }
45 }
46
47 // Si hay una esquina disponible, la toma.
48 for (int i = 0; i < 4; ++i) {
49     if (b.b[corners[i]] == 0) return corners[i];
50 }
51
52 // Bordes
53 int sides[4] = {1, 3, 5, 7};
54 for (int i = 0; i < 4; ++i) {
55     if (b.b[sides[i]] == 0) {
56         return sides[i];

```



```

57         }
58     }
59
60     return -1;
61 }
62
63 string label() { return "rule_based";}
64 };

```

Como se puede ver en la implementación, este jugador es básicamente una mejora del jugador **agresive_t**. Las primeras decisiones de ganar o bloquear en caso de que la jugada esté disponible, son iguales. Luego, en vez de tomar una decisión de forma aleatoria como lo hace el jugador **agresive_t**, se verifican ciertas condiciones específicas del tablero que pueden mejorar la probabilidad de ganar (o de al menos no perder) del jugador **rule_based_t**.

Ejemplo de partida entre **rule_based_t** (X) y **random_t** (O):

```
bash
```

```
rule_based vs random:
```

```
Jugador X juega en la celda 4
```

```
...
. X .
...
```

```
-----
Jugador O juega en la celda 3
```

```
...
O X .
...
```

```
-----
Jugador X juega en la celda 0
```

```
X..
O X .
...
```

```
-----
Jugador O juega en la celda 8
```

```
X..
O X .
.. O
```

```
-----
Jugador X juega en la celda 2
```

```
X . X
```

```
O X .  
.. O
```

Jugador O juega en la celda 5

```
X . X  
O X O  
.. O
```

Jugador X juega en la celda 1

```
X X X  
O X O  
.. O
```

Gana X

Resultados

Para estudiar el comportamiento de los distintos jugadores y las distintas configuraciones entre ellos, se realizaron 1000 partidas entre cada par de jugadores, y se registraron las victorias de cada jugador y los empates. Las figuras a continuación muestran, para un jugador **X** de un cierto tipo fijo, el resultado de las 1000 partidas frente a distintos jugadores **O**.

Por ejemplo, en este primer caso, el jugador **X** es un jugador **dumb_t**, donde solo tiene chances de ganar cuando se enfrenta a otro **dumb_t** (como ya discutimos previamente) y cuando se enfrenta a un **random_t**. En cualquier otro caso, es imposible que este jugador gane, por más que juegue primero.

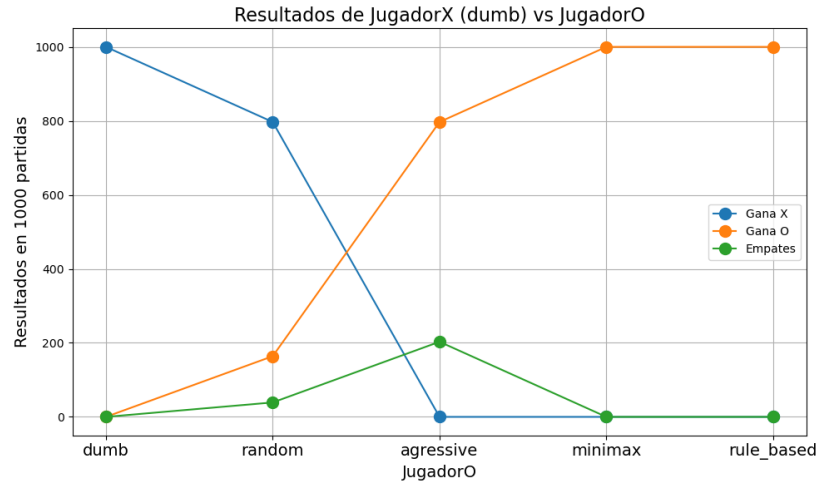


Figure 1: dumb_vs_all

Cuando el jugador X es un jugador `random_t`, es interesante de ver que el problema no es simétrico al ser X siempre el jugador que va primero. Cuando se enfrenta dos `random_t`, X tiene un poco más de chances de ganar. Sin embargo, no tiene chances contra los jugadores que toman alguna decisión inteligente.

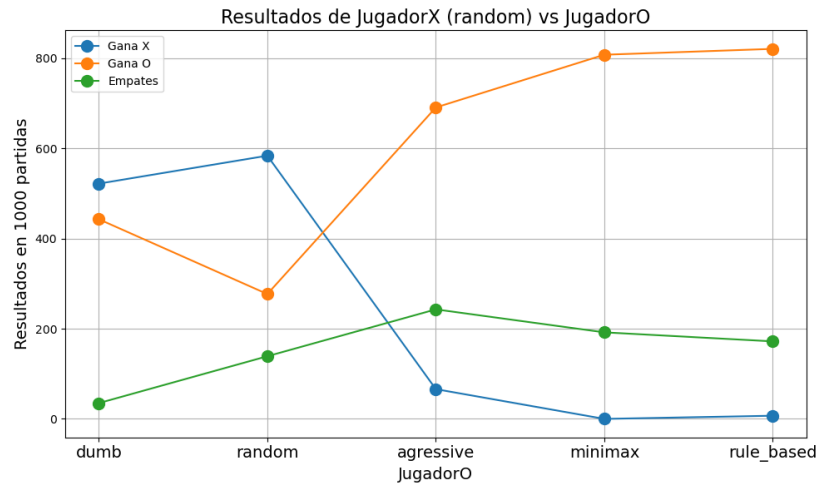


Figure 2: random_vs_all

Cuando ambos son `agresive_t`, es interesante ver que prácticamente siempre gana el jugador X. Por ser éste el que comienza, siempre está un movimiento por delante y lo único que puede hacer el rival, es enfocarse en bloquearlo.

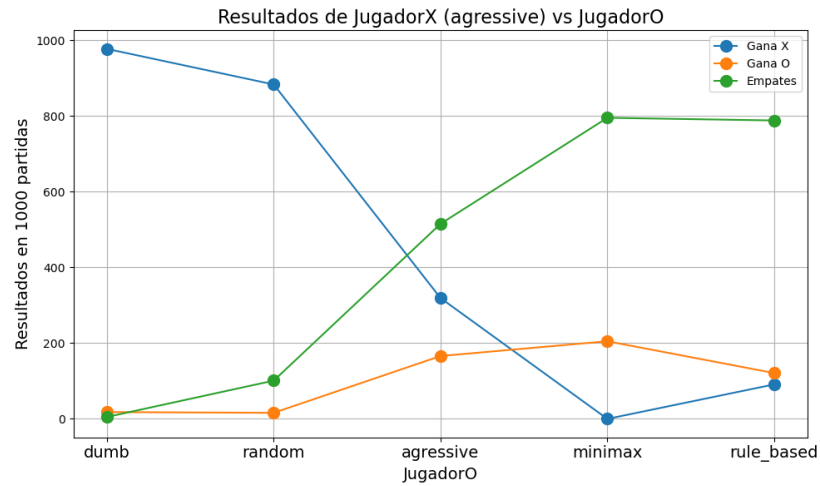


Figure 3: agresive_vs_all

Y como discutimos previamente, en el caso de que el jugador X sea un jugador `minimax_t` o `rule_based_t`, ya se vuelve imposible que el jugador O gane, a lo sumo puede forzar un empate.

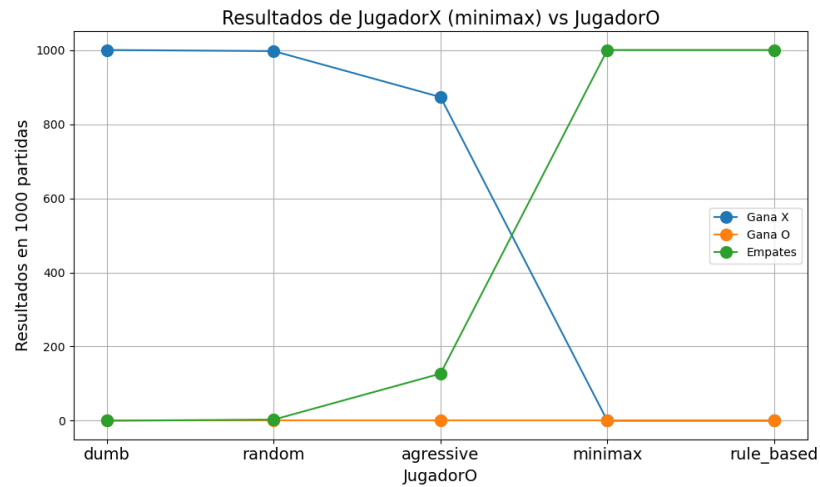


Figure 4: minimax_vs_all

Prestar atención en cómo el `rule_based_t` tiene un desempeño casi idéntico al `minimax_t`, sin necesidad de ser tan compleja su implementación.

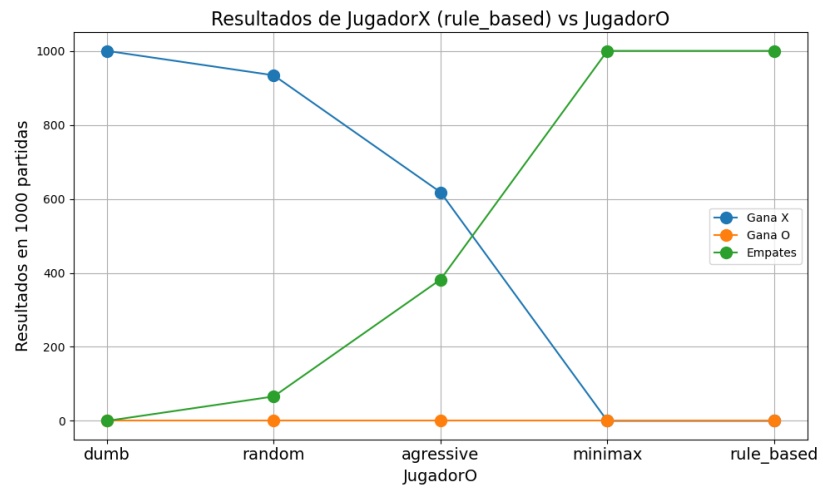


Figure 5: rule_based_vs_all